ROBUST DATA-DRIVEN ANALYSIS FOR ELECTRICITY THEFT ATTACK-RESILIENT POWER GRID

A Project Report submitted for the partial fulfillment of the requirement for the award of the degree of

MASTER OF COMPUTER APPLICATIONS

SUBMITTED
BY

GOVARDHAN REDDICHARLA (Reg No.: 23691F0049)

Under the Guidance of

Dr. Srinivasan J MCA., Ph.D.

Assistant Professor

Department of Computer Applications

DEPARTMENT OF COMPUTER APPLICATIONS

MADANAPALLE INSTITUTE OF TECHNOLOGY & SCIENCE

UGC AUTONOMOUS

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Anantapur)

www.mits.ac.in

2024-2025

MADANAPALLE INSTITUTE OF TECHNOLOGY & SCIENCE

(UGC AUTONOMOUS)

(Approved by AICTE, New Delhi & Affiliated to JNTUA, Anantapur)

DEPARTMENT OF COMPUTER APPLICATIONS

BONAFIDE CERTIFICATE

This is to certify that the project work entitled **"ROBUST DATA-DRIVEN ANALYSIS FOR ELECTRICITY THEFT ATTACK-RESILIENT POWER GRID"** is a Bonafide work carried out by **GOVARDHAN REDDICHARLA**(Roll No.: 23691F0049) submitted in the partial fulfillment of the requirements for the award of the degree of Master of Computer Applications in Madanpalle Institute of Technology & Science, Madanpalle, affiliated to Jawaharlal Nehru Technological University Anantapur, the academic year 2024-2025.

PROJECT GUIDE
Dr. Srinivasan J MCA., Ph.D.
Assistant Professor
Department of Computer Applications

HEAD OF THE DEPARTMENT
Dr. N. Naveen Kumar MCA., Ph.D.
Professor & HOD
Department of Computer Applications

Submitted for viva voce examination held on _____

Internal Examiner
Date:

External Examiner
Date:

MADANAPALLE INSTITUTE OF TECHNOLOGY & SCIENCE

(UGC-AUTONOMOUS INSTITUTION)

Affiliated to JNTUA, Ananthapuramu &Approved by AICTE, New Delhi

NAAC Accredited with A+ Grade, NIRF India Rankings 2021 - Band: 201-250 (Engg.)

NBA Accredited - B.Tech. (CIVIL, CSE, ECE, EEE, MECH), MBA & MCA

Plagiarism Verification Certificate

This is to certify that the MCA Major Project entitled as **"ROBUST DATA-DRIVEN ANALYSIS FOR ELECTRICITY THEFT ATTACK-RESILIENT POWER GRID"** submitted by **Mr. Govardhan Reddicharla (Reg.No.23691F0049)** has been evaluated using **Anti-Plagiarism** Software, **TURNITIN** and based on the analysis report generated by the software, the similarity index is found to be **16 %**.

The following is the **TURNITIN** report for the Major Project consisting of **46** pages.

Department Library Co-Ordinator

DECLARATION

I, GOVARDHAN REDDICHARLA (Roll No.: 23691F0049) hereby declare that the project entitled **"ROBUST DATA-DRIVEN ANALYSIS FOR ELECTRICITY THEFT ATTACK-RESILIENT POWER GRID"** is done by me under the guidance of **Dr. Srinivasan J MCA., Ph.D.** submitted in partial fulfillment of the requirements for the award of degree of Master of Computer Applications at **MADANAPALLE INSTITUTE OF TECHNOLOGY & SCIENCE**, Madanpalle affiliated to Jawaharlal Nehru Technological University Anantapur, during the academic year 2024-2025.This work has not been submitted by anybody towards the award of any degree.

Date:

Place: Madanpalle

Signature of the Student

(R GOVARDHAN)

(Roll No.: 20691F0049)

ACKNOWLEDGEMENT

First, we must thank the almighty who granted me knowledge, wisdom, strength and courage to serve in this world and to carry out my project work in a successfully way.

Next, we have to thank my parents who encouraged me to undergo this MCA course and do this project in a proper way.

We express my sincere thanks to **Dr. N. Vijaya Bhaskar Chowdary Ph.D.** Secretary& Correspondent, Madanpalle Institute of Technology & Science for his continuous encouragement to wards practical education and constant support in all aspects which includes the provision of very good infrastructure facilities in the institute.

It is my duty to thank our Principal, **Dr. C. Yuvaraj M.E., Ph.D.** Madanpalle Institute of technology & science, for his guidance and support at the time of my course and project.

I express my sincere thanks to vice principal PG programs **Dr. R. Kalpana Ph.D.** for her constant support and encouragement during the project.

It is my foremost duty to thank Head of the Department **Dr. N. Naveen Kumar MCA., Ph.D**. who gave me constant support during the project time and continuous encouragement to words the completion of the project successfully.

I thank my faculty guide, **Dr. J. Srinivasan MCA., Ph.D.** for his continuous support by conducting periodical reviews and guidance until the completion of the project in as successful manner.

GOVARDHAN REDDICHARLA(23691F0049)

ABSTRACT

In the era of smart grids and digital energy monitoring, electricity theft remains one of the most critical challenges impacting power distribution systems worldwide. Conventional theft detection techniques are limited by their static thresholds, manual inspections, and inability to adapt to evolving fraud strategies. This project presents a robust, data-driven approach to detecting electricity theft using advanced machine learning techniques. The proposed system utilizes real-world smart meter consumption data and applies preprocessing techniques to clean and balance the data using SMOTE, addressing the inherent class imbalance between normal and fraudulent cases. Feature selection methods are employed to extract the most relevant attributes, enhancing model performance and reducing training complexity. Multiple machine learning models—including Decision Tree, Random Forest, Support Vector Machine (SVM), XGBoost, and Artificial Neural Network (ANN)—are trained and evaluated to classify energy usage behaviour and identify anomalies indicative of theft. Among these, the XGBoost model demonstrated the highest accuracy and recall, making it most suitable for practical deployment. A Flask-based web dashboard is developed to visualize predictions, generate alerts for suspected theft, and offer real-time monitoring for utility administrators. The overall system is scalable, adaptable, and capable of integrating with real-time smart grid infrastructure. This project significantly improves theft detection accuracy while reducing manual workload and financial losses, contributing to the development of a secure and attack-resilient power grid.

Keywords: Electricity Theft Detection, Smart Grid, Machine Learning, XGBoost, SMOTE, Flask Dashboard, Data Imbalance, Feature Selection.

TABLE OF CONTENTS

LIST OF FIGURES

LIST OF ABBREVAITIONS

S.NO.	ABBREVIATED TERM	EXPANSION
1	UML	Unified Modeling Language
2	ER Diagram	Entity Relationship Diagram
3	SMOTE	Synthetic Minority Over Sampling Technique
4	XGBoost	Extreme Gradient Boost

CHAPTER-I

INTRODUCTION

1.1 About the Project

Electricity theft is a growing concern in modern power distribution systems, especially as the shift toward smart grids and automated metering infrastructure (AMI) accelerates. Despite the digital transformation in energy monitoring, illegal electricity consumption remains a persistent challenge that causes significant revenue loss to utility companies and contributes to grid instability. In order to detect sophisticated kinds of fraud, traditional detection techniques—which frequently depend on manual auditing or static threshold-based systems—are no longer enough.

As smart meters generate high-frequency, granular electricity usage data, the volume and complexity of this information demand intelligent analysis techniques. Manual inspection of such massive datasets is not feasible, and static rules can be easily bypassed by experienced fraudsters. Therefore, leveraging data-driven methods, particularly machine learning, has become essential for identifying anomalies in power usage patterns that may indicate theft.

This project aims to develop a robust and scalable electricity theft detection system using advanced machine learning techniques. By analyzing historical smart meter consumption data, the system can classify whether a usage pattern is normal or indicative of fraudulent activity. The use of machine learning allows the model to continuously learn and adapt to new forms of consumption behavior, making it more resilient against evolving theft techniques.

The dataset's extreme imbalance is one of the main obstacles to theft detection. In actual situations, there are considerably more valid records of electricity use than there are thefts. Biassed models that are unable to correctly identify the minority class may result from this imbalance. The Synthetic Minority Over-Sampling Technique (SMOTE), which creates synthetic theft samples to enhance model learning and detection performance, is incorporated into the project to remedy issue.

The raw data collected from smart meters often contains inconsistencies, missing values, and noise. Therefore, the project includes a preprocessing phase where the dataset is cleaned, normalized, and prepared for further analysis. Preprocessing also includes feature extraction and selection techniques, which reduce dimensionality and retain only the most relevant attributes for training the model.

This research investigates five distinct machine learning models: Artificial Neural Network (ANN), Support Vector Machine (SVM), Extreme Gradient Boosting (XGBoost), Random Forest (RF), and Decision Tree (DT). The processed dataset is used to train each of these models, and common classification metrics including accuracy, precision, recall, and F1-score are used to assess each model. The best model for identifying electricity theft is chosen by comparing its performances.

The top-performing model was XGBoost, which provided a good balance between recall and accuracy. It is quite successful in handling noisy data and identifying minute trends in fraudulent consumption behavior thanks to its ensemble-based boosting approach and regularization approaches. The output of the model is then utilized to confidently highlight possible theft cases.

To make the solution more practical and accessible, a Flask-based web dashboard is integrated into the system. This dashboard allows utility providers and administrators to view real-time predictions, receive theft alerts, and visualize overall consumption trends. It serves as a user-friendly interface that bridges the technical model backend with the operational needs of grid operators.

The modular structure of the project ensures that each component—preprocessing, feature selection, model training, and deployment—can be scaled independently or replaced without affecting the overall workflow. This enhances the flexibility and adaptability of the system in real-world power distribution environments.

In conclusion, by providing a clever, effective, and scalable method for detecting electricity theft, our study tackles a significant issue in contemporary energy systems. It demonstrates how machine learning can be used practically in a high-impact field and emphasizes crucial reliable data.

CHAPTER-II

SYSTEM ANALYSIS

2.1 Existing System

Current electricity theft detection systems used by many utility providers are often based on manual inspections, static thresholds, or rule-based logic. These traditional approaches rely heavily on human expertise to identify suspicious consumption patterns by comparing meter readings over time or against regional averages. While these techniques may detect obvious cases of tampering or meter bypassing, they are inefficient. As electricity consumption grows and customer bases expand, such manual methods fail to scale.

Another commonly used approach involves setting fixed consumption limits or thresholds to flag irregular behavior. However, this method assumes that electricity usage follows a consistent and predictable pattern across all consumers. In reality, electricity consumption is highly dynamic and influenced by factors like season, appliance usage, household size, and lifestyle changes. Rigid threshold systems often produce false positives or fail to detect sophisticated theft, where fraudsters manipulate usage just below detection levels.

In some implementations, rule-based detection systems are enhanced with basic statistical analysis or clustering techniques. While these provide marginal improvements, they still lack the adaptability required for real-world scenarios. Fraudsters frequently evolve their tactics, exploiting blind spots in fixed logic. Static systems cannot learn from new patterns or adapt to emerging threats, resulting in outdated models that do not reflect current fraudulent behaviours.

Furthermore, existing systems rarely account for class imbalance in electricity consumption datasets. Theft instances are much rarer compared to normal consumption, which leads to detection models. This results in low recall, meaning many theft cases go undetected. Most legacy systems are not equipped to handle imbalanced data, high-dimensional features, or noisy inputs, limiting their detection accuracy.

Overall, existing electricity theft detection systems lack intelligence, adaptability, and automation. They do not utilize the full potential of data-driven insights from smart meters,

making them inefficient in large-scale smart grid deployments.

Demerits:

- Traditional systems generate frequent false positives and fail to accurately detect sophisticated electricity theft behaviours.
- They lack adaptability to evolving fraud techniques, making them ineffective in dynamic power usage environments.
- Existing methods do not support real-time detection or alert generation, causing delays in identifying theft incidents.
- They perform poorly on imbalanced datasets, often missing actual theft cases due to biased detection logic.

2.2 Proposed System

The suggested approach uses machine learning models to detect electricity theft in an intelligent, data-driven manner. To find unusual usage trends, it examines consumption data from smart meters. This eliminates the need for manual inspections or fixed rule-based systems. The approach ensures high accuracy in detecting both obvious and hidden theft behavior. To overcome class imbalance issues, the system integrates the SMOTE algorithm. SMOTE generates synthetic theft records, improving minority class representation. Data preprocessing is applied to clean, normalize, and prepare the dataset. This ensures better learning and performance across all models.

Decision Tree, Random Forest, SVM, XGBoost, and ANN are the five machine learning models that are trained and assessed. Metrics including accuracy, precision, recall, and F1-score are used to evaluate each model. XGBoost delivers the best results in theft detection reliability. Feature selection techniques further improve model efficiency and reduce overfitting.

A Flask-based dashboard is developed for real-time theft monitoring and visualization. The interface allows utility administrators to view predictions and receive alerts. It bridges model output with operational insights for quick action. The dashboard enhances user experience and system deployability.

Overall, the system is scalable, adaptable, and suitable for real-time smart grid environments. It improves theft detection accuracy while reducing false alarms. By automating the entire process, it reduces manual workload. This system contributes to building a theft-resilient and secure power grid.

Merits:

- Detects electricity theft accurately using machine learning-based classification.
- Balances imbalanced data effectively using the SMOTE technique.
- Enables real-time theft alerts through a Flask-based web dashboard.
- Offers a scalable and modular framework for smart grid deployment.

2.3 HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements

- **Processor:** Intel Core i3 or higher (i5 and i7 have been tested for best performance).
- **RAM:** 4 GB at minimum (for model training, 8 GB or more is advised).
- **Storage:** SSD/HDD 256 GB (SSD is advised for quicker I/O operations).
- **System Architecture:** 64-bit Operating System
- **Display:** Minimum 1366x768 resolution (Recommended: Full HD)

Software Requirements

- **O S:** Windows 10, Windows 11, and Linux (Deployment is best done with Ubuntu).
- **Programming Language:** Python 3.11
- **IDE/Editor:** Visual Studio Code, Flask-app

2.4 Feasibility Study

It evaluates different factors such as cost, technical readiness, and social impact to ensure that the project is both practical and beneficial. This project was assessed under three key dimensions below:

Economic Feasibility

The proposed system economically feasible as it utilizes open-source tools like Python, Flask, and Scikit-learn, eliminating software licensing costs. It reduces manual inspection efforts and financial losses caused by electricity theft. The hardware requirements are modest, and implementation costs are minimal. Overall, the system offers a cost-effective, scalable solution for utility providers.

Technical Feasibility

Technically, the project is viable as it employs stable, widely adopted technologies such as Python, XGBoost, and Flask. The machine learning models run efficiently on commonly available systems with basic hardware configurations. Data handling, preprocessing, and real-time dashboard integration are all technically achievable. The modular architecture allows for easy maintenance and future scalability.

Social Feasibility

Socially, the system is well-accepted as it promotes fairness in billing by detecting theft accurately. It reduces the burden on honest consumers and fosters accountability among users. The dashboard interface is simple and user-friendly, requiring little training for utility staff. Its real-time alert system enhances transparency between providers and consumers.

CHAPTER-3

SYSTEM DESIGN

3.1 MODULE DESCRIPTION:

1. Admin Login:

The system includes a secure admin login interface where authorized personnel (utility staff or analysts) can log in and access the monitoring dashboard. The login system ensures that only authenticated users can view or manage prediction results and theft alerts.

2. Data Upload:

Admins can upload smart meter electricity consumption datasets collected from various regions or customers. The uploaded dataset should be in CSV format, typically containing features like meter ID, timestamp, consumption values, and customer ID.

3. Data Preprocessing:

Once uploaded, the data undergoes preprocessing where missing values are handled, outliers are removed, and normalization is applied. If the dataset is imbalanced (i.e., theft records are fewer), SMOTE is used to synthetically balance the dataset, improving model training and detection reliability.

4. Feature Selection:

Relevant features from the pre-processed data are extracted based on importance using statistical techniques and model-based evaluations. This step helps reduce dimensionality, eliminates noisy or irrelevant features, and enhances overall model performance.

5. Model Selection and Training:

Decision Tree, Random Forest, Support Vector Machine, XGBoost, and Artificial Neural Network are among the machine learning methods that the system supports. The pre-processed dataset can be used by users to train the model, and the system will assess performance using F1-score, accuracy, precision, and recall.

6. Theft Detection and Prediction:

After training, the system can predict whether a particular consumption pattern is normal or fraudulent. Based on the model output, the system will classify the input as “Theft” or “No Theft,” along with a confidence score or probability.

7. Result History:

All previous prediction results are stored and can be accessed via the admin panel. This module provides a historical log of inputs, predictions, timestamps, and model performance metrics for further analysis.

8. Dashboard Visualization:

A Flask-based web dashboard provides real-time visualization of results. It displays graphs, prediction outcomes, and alerts when suspicious consumption behavior is detected. The dashboard is interactive, allowing utility providers to take informed actions quickly.

3.2 ER Diagram

Using an Entity Relationship Diagram (ER Diagram), an entity–relationship model (ER model) uses a diagram to explain the structure of a database. An ER model is a database design or blueprint that can be used to create a database in the future. The entity set and relationship set are the two primary parts of the E-R model.

Entity-Relationship (ER) illustrates logical structure of the electricity theft detection system by modelling entities such as User, Smart Meter, Consumption Data, Machine Learning Model, and Prediction. It shows how each user is associated with a smart meter, which records consumption data that is analyzed by trained models to detect theft. This diagram helps in understanding the system's data flow and the relationships between various functional components before actual development.

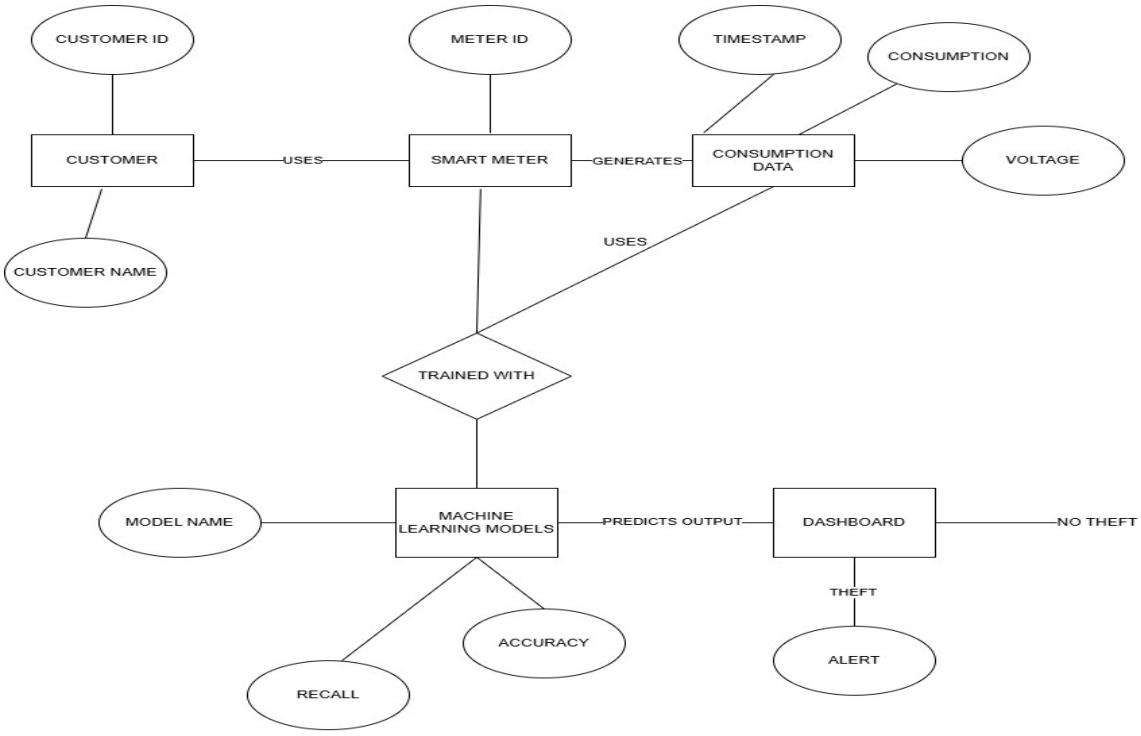


Fig 3.2.1: ER Diagram

3.3 UML DIAGRAMS

These diagrams are used to visually represents structure and behavior of a software system. They help in understanding the architecture, components, user interactions, and system workflows in a standardized way. In this project, UML diagrams are used to design and analyze how different modules of the electricity theft detection system interact with each other and with the end user.

The UML diagrams used in this system include:

- **Use Case Diagram** – Represents user interactions and system functionalities.
- **Activity Diagram** – Describes the workflow and sequence of operations.
- **Sequence Diagram** – The order of messages between system components is displayed via a sequence diagram.
- **Class Diagram** – Shows the classes, characteristics, and connections inside the system.
- **Component Diagram** – Shows the application's modular structure.
- **Deployment Diagram** – Shows the physical deployment of system components on hardware nodes.

These diagrams help in validating the system design before implementation, ensuring clarity, modularity, and better communication among developers and stakeholders.

3.3.1 ACTIVITY DIAGRAM:

The activity diagram illustrates the workflow of the system from dataset upload to theft detection and result visualization. It captures each step including preprocessing, model selection, training, and prediction generation. This helps represent the dynamic behavior and flow of control within the electricity theft detection system.

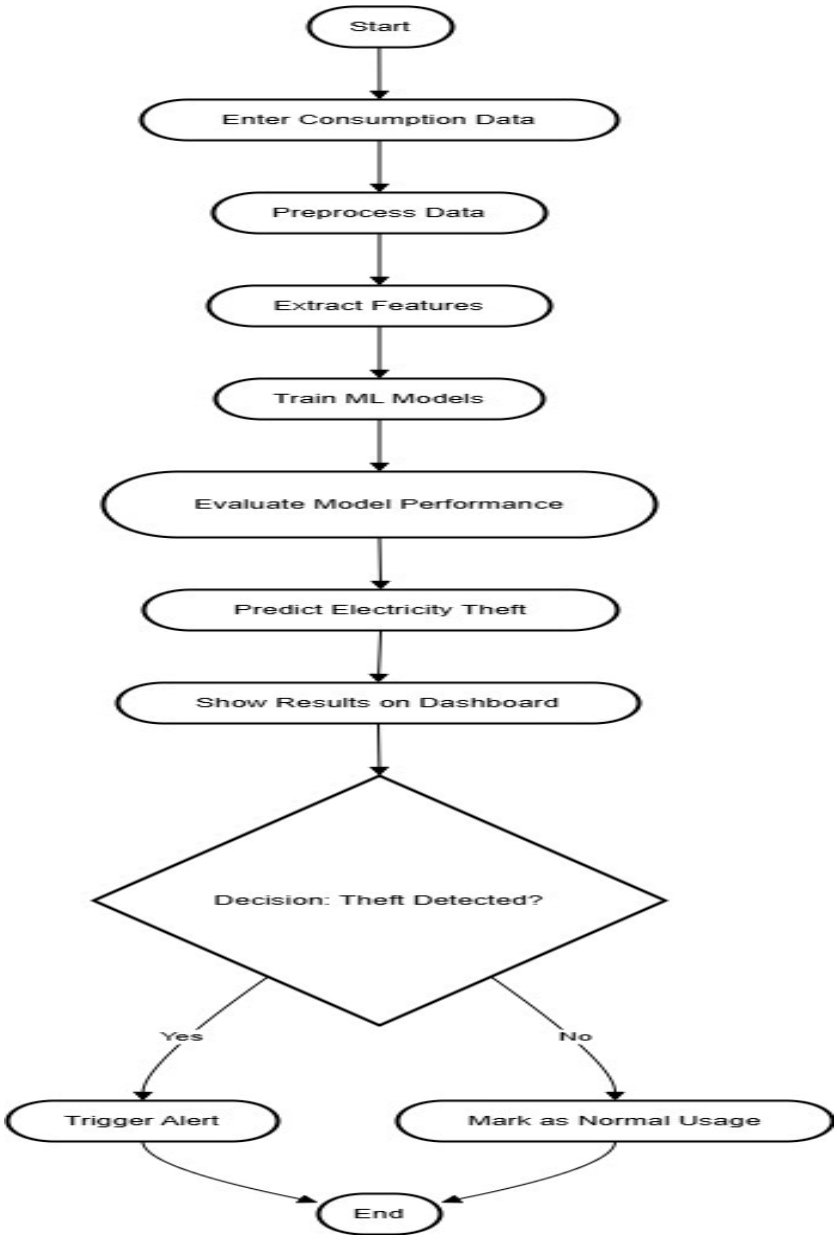


Fig 3.3.1: Activity Diagram

3.3.2 USECASE DIAGRAM:

The use case diagram showcases the interaction between the system and its main actor, the Admin or Utility Operator, who performs core functions. These include uploading the dataset, preprocessing data, selecting and training machine learning models, detecting theft, and viewing prediction results. It helps in understanding the functional boundaries of the system and user-system interaction flow.

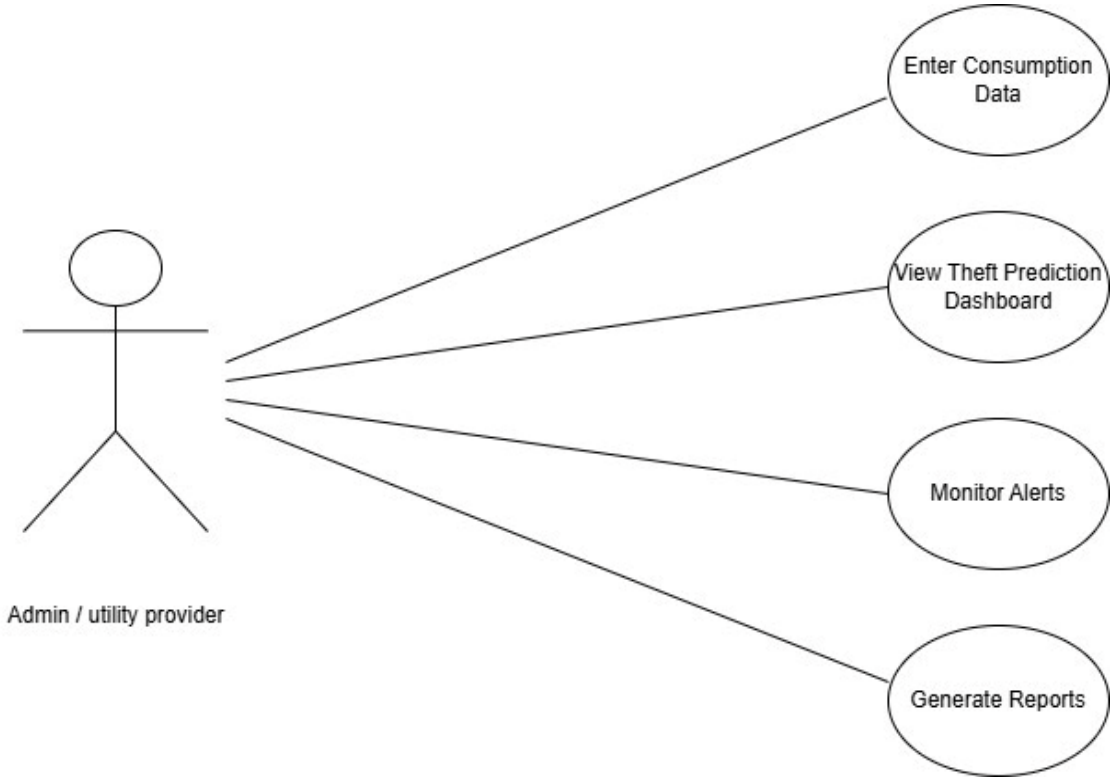


Fig 3.3.2: Use Case Diagram

3.3.3 SEQUENCE DIAGRAM:

Represents chronological interactions among user and system components. It begins with the admin uploading the dataset, followed by preprocessing, feature selection, and model training. Once trained, the model returns the theft detection result after receiving input data for prediction. Finally, the system displays the outcome through the dashboard for the user to review.

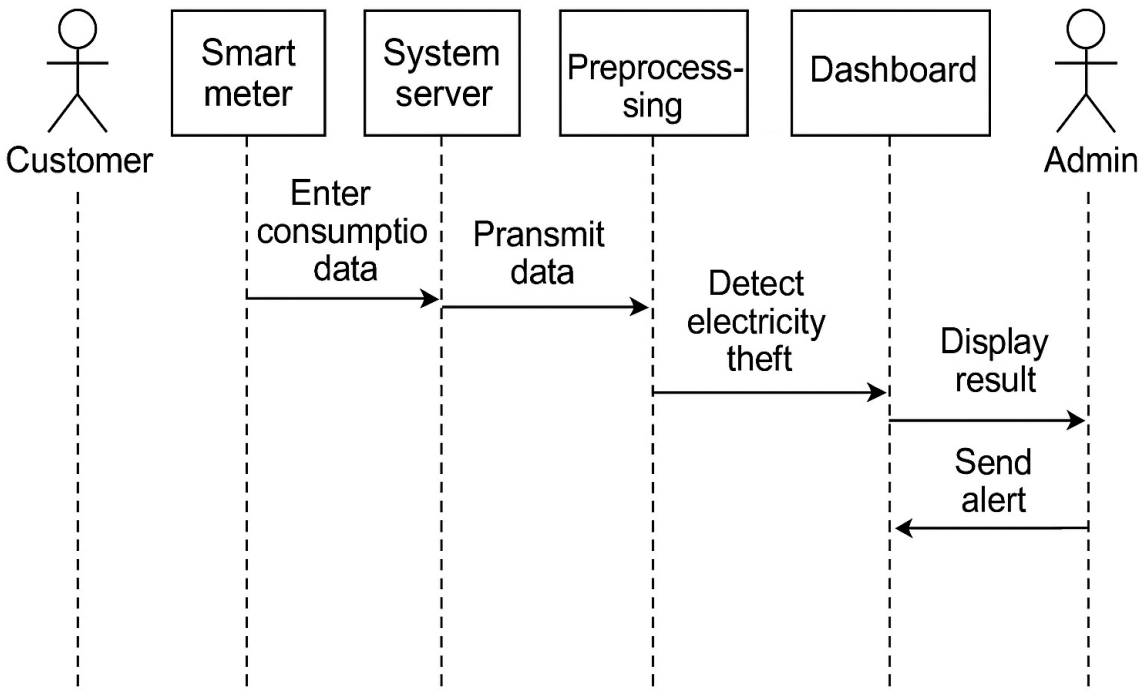


Fig 3.3.3: Sequence Diagram

3.3.4 DEPLOYMENT DIAGRAM:

It includes nodes like the user system (admin interface), application server (model processing), and database server (data storage). The admin uploads data via a web interface hosted on the application server. The trained machine learning models run on this server, interacting with the database for input/output. This setup supports real-time processing, prediction, and dashboard visualization in a smart grid context.

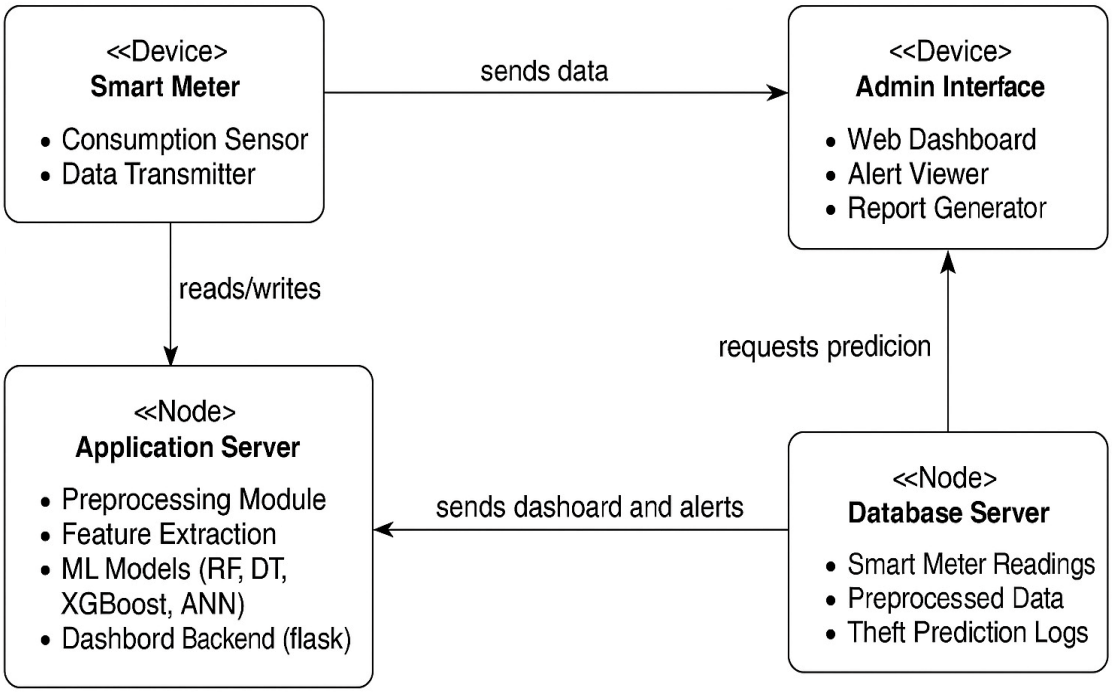


Fig 3.3.4: Deployment Diagram

3.3.5 COLLABORATION DIAGRAM:

It emphasizes the relationships and message flow between components like the user interface, preprocessing module, machine learning engine, and result generator. The admin initiates the process by uploading data, which is then cleaned and passed to the model for training and prediction. Results are returned once the prediction is made and shown on the dashboard. This diagram highlights object interactions and the structural organization behind functional flow.

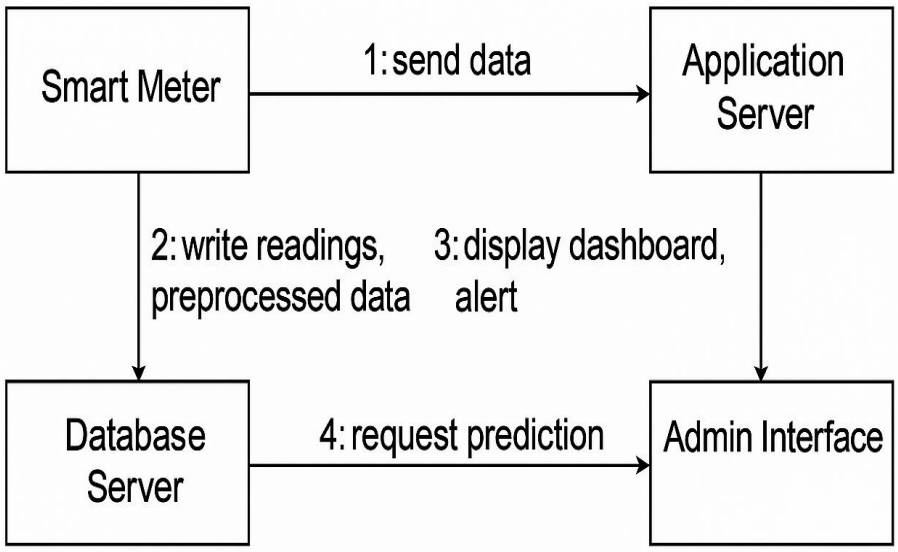


Fig 3.3.5: Collaboration Diagram

3.3.6 CLASS DIAGRAM:

By outlining the system's classes, characteristics, methods, and relationships, the class diagram illustrates its static structure. Among the important classes are User, SmartMeter, Consumption Data, ML Model, and Theft Prediction. The User class is associated with a SmartMeter, which records and stores electricity usage in the Consumption Data class. The ML Model class contains methods for training, prediction, and evaluation. Theft Prediction stores model results and flags suspicious activity. Relationships like association and dependency connect these classes to reflect system logic.

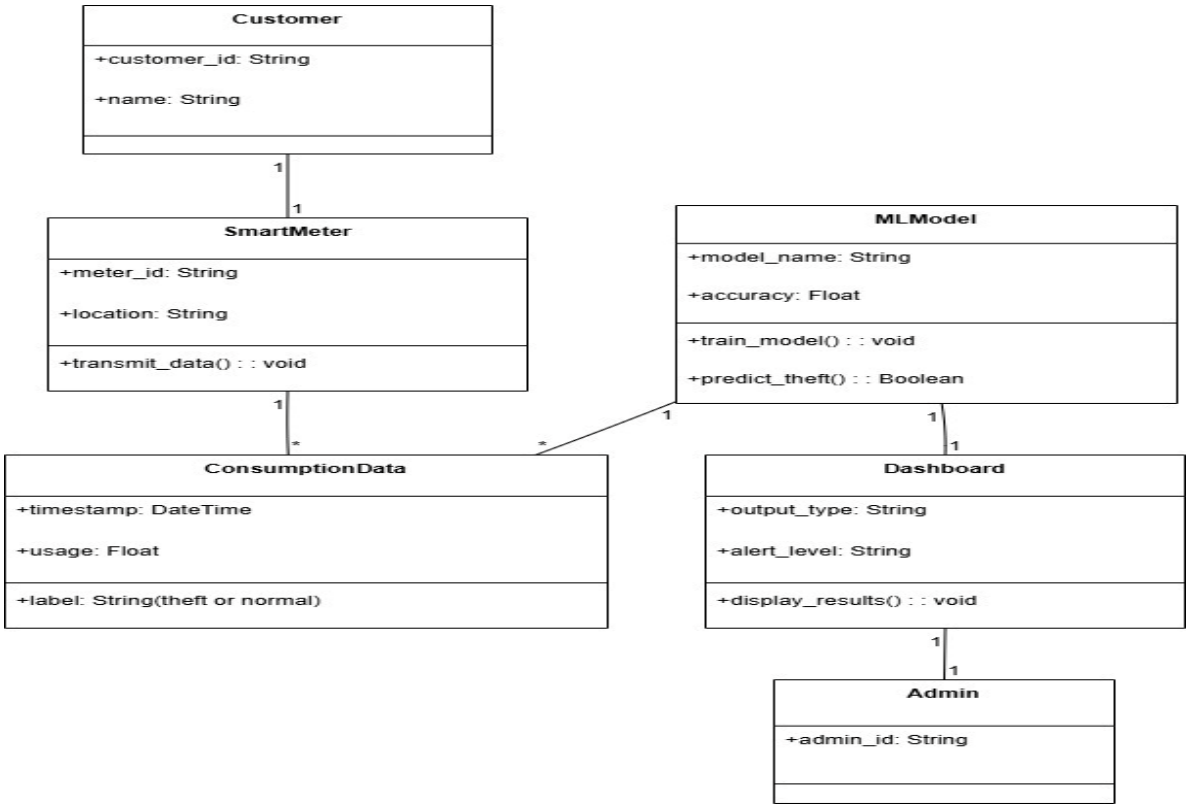


Fig 3.3.6: Class Diagram

3.3.7 COMPONENT DIAGRAM:

The system's modular structure is depicted in the component diagram and how different components interact. Major components include the Data Preprocessing Module, Feature Selection Module, Machine Learning Module, Theft Detection Engine, and Flask Dashboard Interface. Each component handles a specific function and communicates with others through defined interfaces. The ML module connects with the database to access training and prediction data. This modular design enhances scalability, maintainability, and real-time deployment of the system.

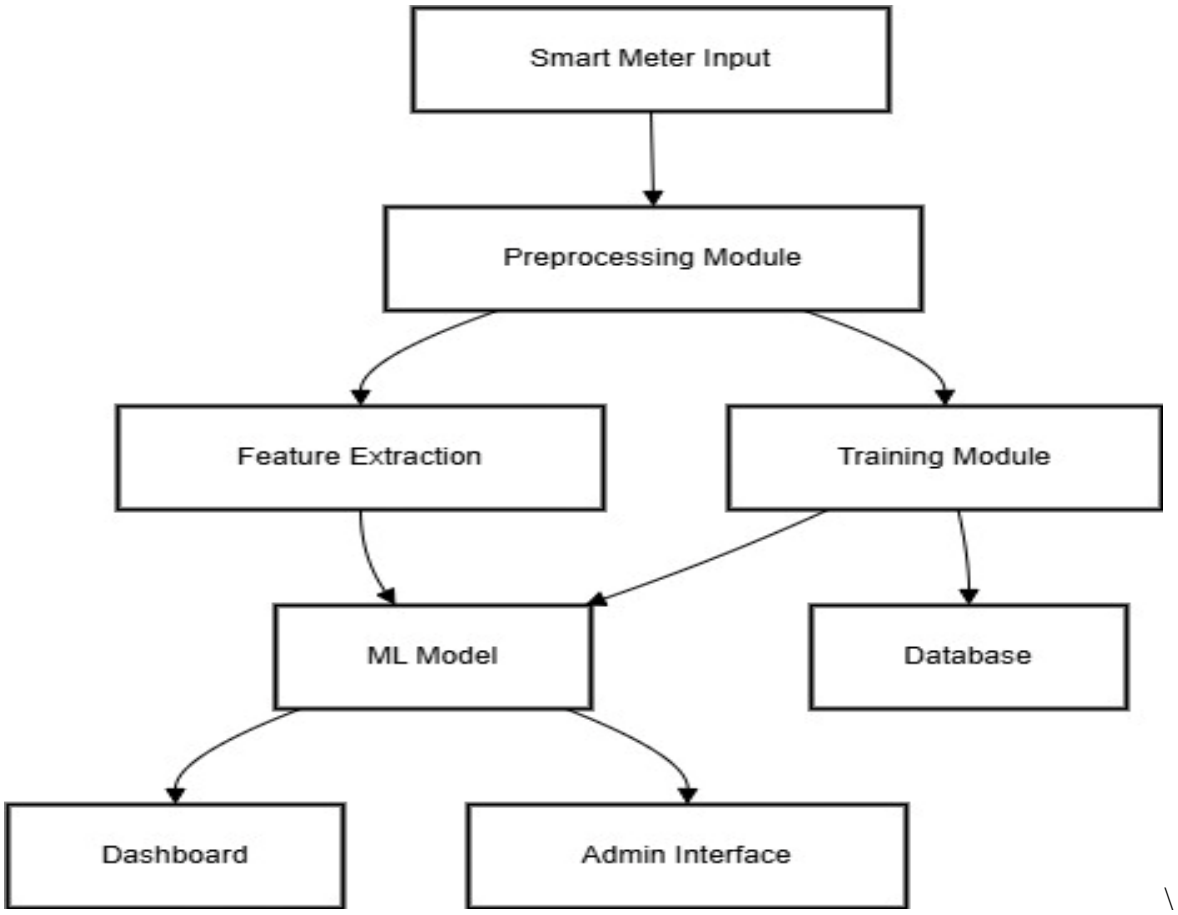


Fig 3.3.7: Component Diagram

CHAPTER-4

SYSETM IMPLEMENTATION

4.1 Language Selection – Python

Overview of Python

Python is a general-purpose, high-level, open-source programming language that is frequently used in web development, machine learning, and data science. Numerous programming paradigms, including procedural, object-oriented, and functional programming, are supported. Python's extensive library ecosystem and easily understood syntax make it an ideal choice for beginners. For this undertaking, Python 3.11 is used as the core language for implementing electricity theft detection and web-based monitoring.

4.1.1 History

Python was developed in the late 1980s and was initially used by Guido van Rossum at CWI in the Netherlands in December 1989 as an alternative to ABC that could manage exceptions and communicate with the Amoeba operating system. Since he continues to play a crucial role in shaping the direction of the language, the Python community has named Van Rossum the principal author of Python. The introduction of Python 2.0 on October 16, 2000, included a number of noteworthy new features, such as support for Unicode and a garbage collector that detects cycles for memory management (in addition to reference counting). The biggest shift was in the development process itself, transitioning to a more community-supported and open approach. On December 3, 2008, Python 3.0, a major, backwards-incompatible update, was released following rigorous testing.

A large number of its primary functions have also been backported to Python 2.6 and 2.7, which are no longer supported but are nonetheless backwards compatible.

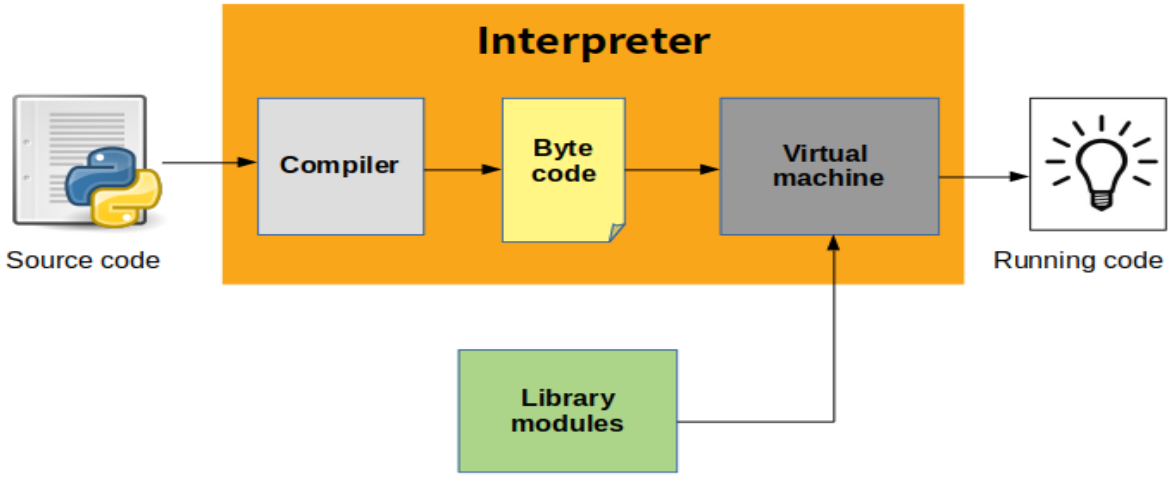
4.1.2 Why Python for This Project?

Python is highly compatible with data science libraries and frameworks, making it ideal for building machine learning models. It provides rich support for data manipulation, visualization, model training, and deployment. Since Machine learning is used in the suggested system (XGBoost, RF, ANN, etc.), data preprocessing (Pandas, NumPy), and web dashboard development (Flask), Python is the most efficient and scalable choice.

Python offers a great interactive environment and is widely known for its simplicity and versatility. It is an object-oriented, interactive, high-level, interpreted programming language that promotes clean and readable code. Unlike many other programming languages that rely heavily on symbols, Python uses English-like keywords, making it more intuitive and accessible for beginners and professionals alike.

Python is an interpreted language, which means that the interpreter processes it at runtime without requiring compilation beforehand. Similar to languages like PHP and PERL, this functionality enables rapid code testing and debugging. Additionally, is **interactive**, enabling developers to directly execute commands and view results in real-time through the Python prompt.

Python also supports the **object-oriented programming paradigm**, allowing developers to encapsulate functionality within classes and objects, which improves code modularity and reusability. Its beginner-friendly nature and rich ecosystem of libraries make Python an excellent choice for building a variety of uses, ranging from basic scripts for data analysis to sophisticated web apps and machine learning systems.



4.1.3 Python Key Features Used in This Project

- **Interpreted Language:** Python code is run without compilation, line by line.
- **Object-Oriented:** Used for structuring machine learning components.
- **Extensive Libraries:** Libraries like scikit-learn, XGBoost, Pandas, NumPy, and Flask make implementation seamless.
- **Database Integration:** Python supports MySQL integration via MySQL. Connector.

4.1.4 Python Standard Data Types Used

- **Numbers**: For calculating electricity usage metrics and statistical features.
- **Strings**: For handling textual metadata like meter IDs and user types.
- **Lists & Tuples**: Used to store sequences of inputs and outputs from models.
- **Dictionaries**: Useful for storing key-value pairs, such as user details and prediction outcomes.

4.1.5 Python Libraries Used

1. NumPy

NumPy (Numerical Python) is used for efficient numerical computations and array operations. It supports operations in linear algebra, mathematical functions, and multi-dimensional arrays. In this project, NumPy is used extensively for handling datasets, normalizing values, and manipulating numerical data during preprocessing and model training phases.

2. Pandas

Built on top of NumPy, Pandas is a robust data manipulation and analysis package. is used to load, explore, clean, and preprocess the dataset. In this project, Pandas helps in reading CSV files, handling missing values, removing duplicates, and preparing structured data before passing it to machine learning models.

3. Scikit-learn (sklearn)

A complete machine learning library, Scikit-learn offers evaluation metrics, preprocessing tools, and implementations of numerous algorithms. Models like SVM, Random Forest, and Decision Tree are implemented in this project. Additionally, it provides tools for hyperparameter adjustment, confusion matrix generation, accuracy calculation, and train-test separation.

4. XGBoost

For supervised learning situations, XGBoost (Extreme Gradient Boosting) is a sophisticated and well-optimized gradient boosting toolkit. Its speed and performance are well-known. In this project, XGBoost is applied to achieve high accuracy and recall in theft detection, making it one of the best-performing models.

5. TensorFlow / Keras

It supports building deep learning models, defining hidden layers, and training networks on large datasets.

6. Imbalanced-learn (imblearn)

Resampling methods created especially to deal with imbalanced datasets are available in the imbalanced-learn library. In order to balance the dataset and enhance model performance on minority class predictions, the project makes use of the SMOTE (Synthetic Minority Over-sampling Technique) function from this library to create synthetic cases of stolen data.

7. Flask

Python, used building the project's web-based interface. It is used to create an interactive dashboard that allows the admin to upload data, view predictions, and monitor theft alerts in real time. Flask connects the backend machine learning logic with the frontend interface.

8. Matplotlib & Seaborn

Data visualization makes use of these libraries. Basic charting capabilities are provided by Matplotlib, while Seaborn expands upon top of it to produce more visually appealing graphs. In this project, they are used to plot confusion matrices, feature importance charts, and other performance-related graphs to understand model behavior.

9. Joblib

Joblib is a utility library for saving and loading trained models. it serializes models after training so they can be reused in the Flask app without retraining, making the system faster and more efficient.

Python Modules and Concepts Used

- **Modules**: Python files (.py) are organized as modules for feature selection, training, and visualization.
- **Classes and Objects**: Used for model encapsulation and organizing system logic.
- **Functions**: Defined for preprocessing, training, and result prediction.
- **Interactive Mode**: Used during development for testing individual components.

- **Script Mode**: Used for running the finalized system and dashboard.

4.2 Database Integration

This project uses **MySQL** as the backend for storing user credentials and prediction history. The MySQL. connector module enables seamless integration between Python and MySQL databases. Data related to meter readings, user input, and prediction results are stored and retrieved from MySQL tables securely and efficiently.

4.2.1 Use of MySQL in the Project

MySQL is used in this project as the backend manage and store information, credentials, consumption data records, prediction results, and theft history. It ensures data persistence, reliability, and secure access to sensitive information related to electricity usage and fraud detection.

Python connects to the MySQL database using the MySQL. connector module, which supports smooth data transfer between the machine learning system and the database. After preprocessing and prediction, the system logs model outputs—such as predicted theft status, confidence scores, and timestamps—into the database. This allows the admin to view historical prediction results through the Flask-based dashboard.

The database is also responsible for managing user authentication, including storing admin login credentials and controlling access to system functionalities. MySQL's structured query language (SQL) enables fast querying, updating, and retrieving of data for display on the web interface.

Overall, MySQL a secure, structured, scalable backend for electricity theft detection system, supporting both data analysis and real-time monitoring features.

4.3 Machine Learning

ML models analyse patterns in historical smart meter data to distinguish between normal consumption behavior and potentially fraudulent usage. This data-driven approach is especially effective in detecting complex or evolving theft strategies that traditional rule-based systems often miss.

In this project, several supervised learning algorithms were implemented to classify electricity usage patterns into "Theft" and "No Theft" categories. Supervised learning involves training models on labelled datasets, where the input features are associated with known outcomes. Once trained, the models can accurately use known patterns to predict the class of new, unseen data.

4.3.1 Machine Learning Algorithms

1. Decision Tree (DT)

It starts from root node and branches out based on logical conditions until it reaches a final decision at a leaf node. It is easy to interpret, handles both numerical and categorical data, and works well for classification tasks. However, deep trees may overfit the data, reducing generalization to unseen cases.

2. Random Forest (RF)

Several decision trees are combined in Random Forest, an ensemble learning technique, to generate predictions that are more reliable and accurate. It uses random subsets of features and data to generate each tree. then aggregates the results through majority voting. This reduces the risk of overfitting that occurs in single trees and improves performance on noisy data. Random Forest is highly robust, scalable, and works effectively. widely used due to its simplicity and high accuracy.

3. Support Vector Machine (SVM)

SVM is a potent supervised learning technique that finds the best hyperplane to divide the dataset's classes. It transforms the input space using kernel functions and performs well with both linear and non-linear data. SVM aims to increase classification confidence by optimizing the margin between classes.

4. XGBoost (Extreme Gradient Boosting)

A sophisticated machine learning method built for speed and performance, XGBoost is based on the gradient boosting framework. It constructs a sequence of decision trees, each of which aims to fix the mistakes of the one before it. XGBoost can effectively manage missing data and minimize overfitting by using regularization techniques.

5. Artificial Neural Network (ANN)

The human brain served as the inspiration for the artificial neural network (ANN), a deep learning model made up of layers of connected neurons that process and learn from input data. It is particularly helpful for identifying subtle patterns in big datasets and can capture intricate, non-linear relationships. During training, weights are changed using methods like gradient descent and backpropagation. ANNs need more training data and processing power than conventional models. In this system, ANN was used to evaluate deep learning performance for theft classification.

SMOTE – Synthetic Minority Over-sampling Technique

There is a class imbalance in real-world electricity usage data since theft instances are far lower than typical cases. Machine learning models become biased toward the dominant class as a result of this imbalance, which makes it difficult to detect real theft. SMOTE (Synthetic Minority Over-sampling Technique) was used to get around this. SMOTE interpolates between current theft cases to create synthetic samples of the minority class (theft). By balancing the dataset without sacrificing crucial information, our method enhances recall and classification performance while enabling the machine to learn thieving patterns more successfully.

Hyperparameter Tuning

A crucial step in maximizing the effectiveness of machine learning models is hyperparameter tuning. The number of estimators, learning rate, maximum depth, kernel type, and activation functions are among the particular hyperparameters that are present in each of the algorithms.

4.4 SCREEN SHOTS

The figure depicts the main window of the application which is displayed through executing the application file.

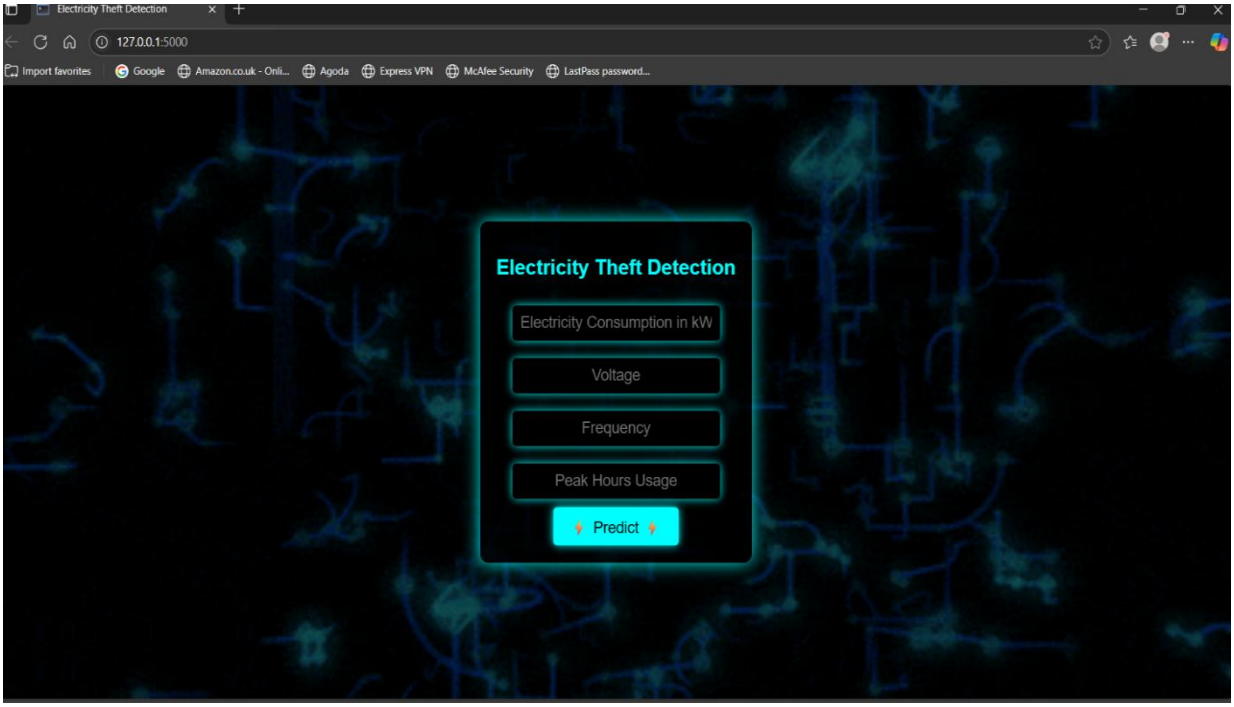


Fig 4.4.1: Main window

The homepage of the Electricity Theft Detection system serves as the user-facing entry point for inputting electricity usage data and initiating predictions. Designed with a modern, tech-themed dark background and glowing electric elements, the interface immediately signals its relevance to energy systems and smart grid technology.

key input fields:

- Electricity Consumption in kW
- Voltage
- Frequency
- Peak Hours Usage

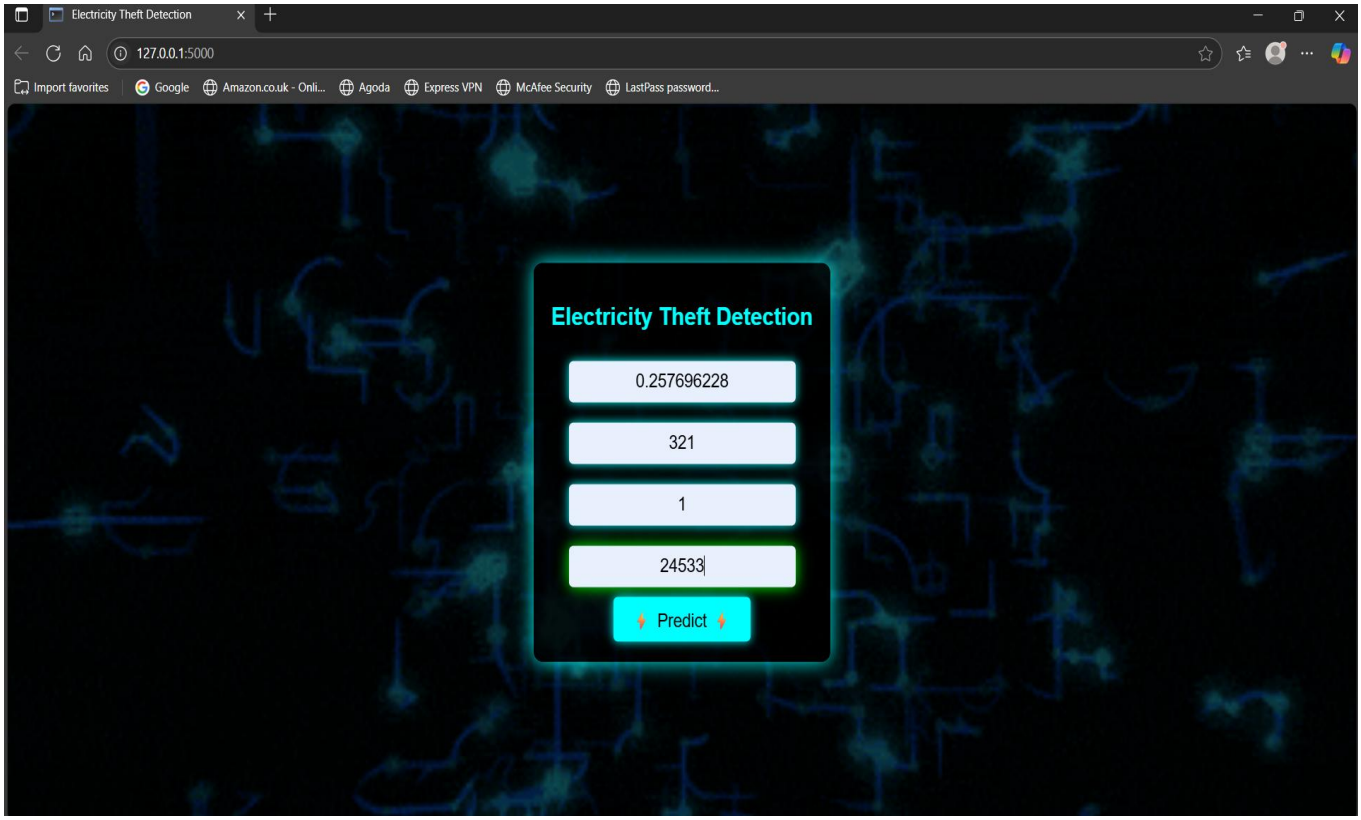


Fig 4.4.2: Shows Textbox Values

The figure depicts the activity is user enter the data in the textbox , represents the **active prediction stage** of the Electricity Theft Detection system. At this point, the user has already entered specific numeric values into each input field on the homepage. The entered values correspond to four key parameters:

- **Electricity Consumption in kW** (0.257696228)
- **Voltage** (321)
- **Frequency** (1)
- **Peak Hours Usage** (24533)

Triggers backend Flask API to process the inputs and return a prediction result (such as "Theft" or "No Theft").

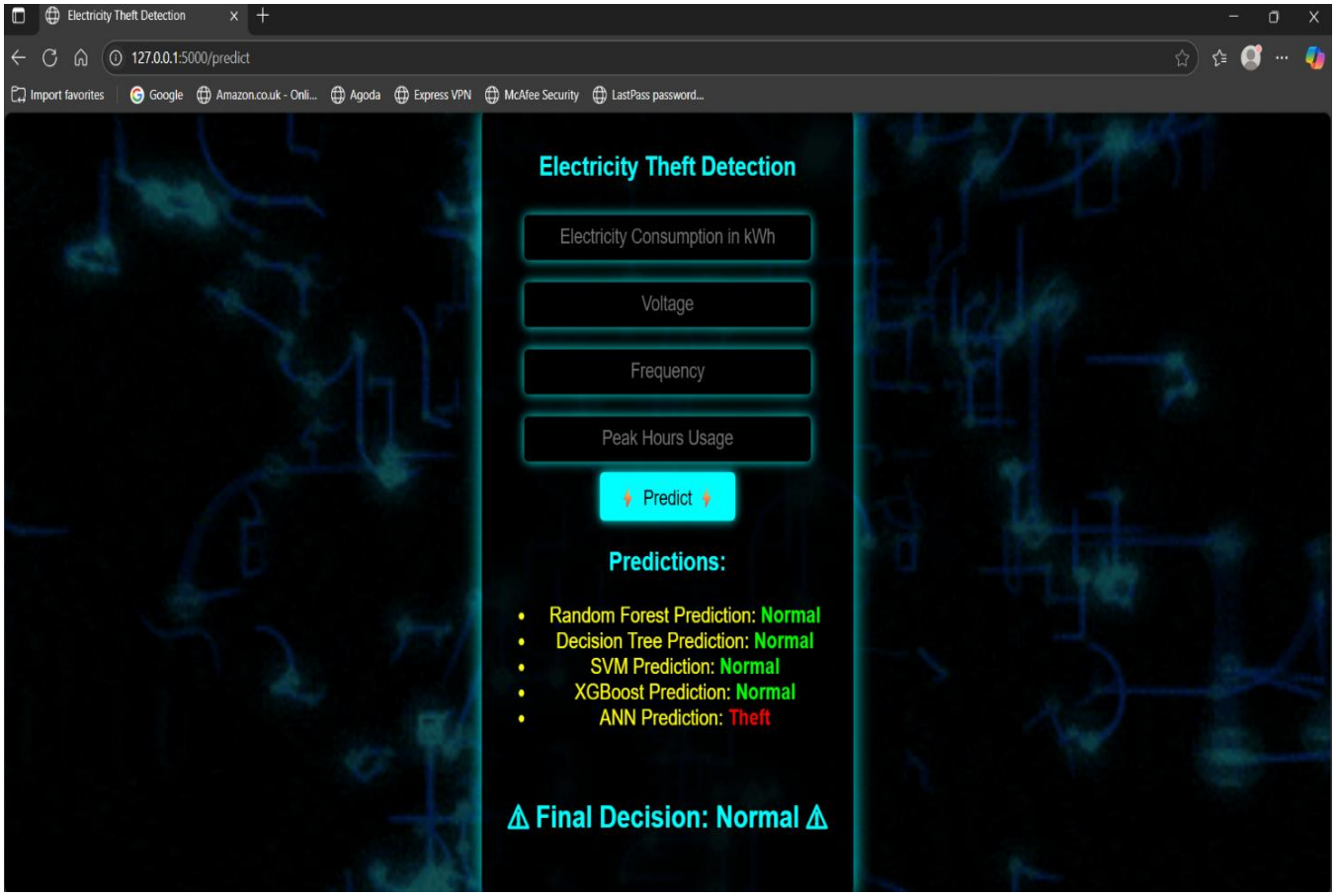


Fig 4.4.3: No Electricity Theft

This image represents the **prediction results page** of the *Electricity Theft Detection System*. Once the user provides input values such as:

- **Electricity Consumption in kWh**
- **Voltage**
- **Frequency**
- **Peak Hours Usage**

and clicks on the **"Predict "** button, the system displays the predictions from all the machine learning models integrated into the backend.

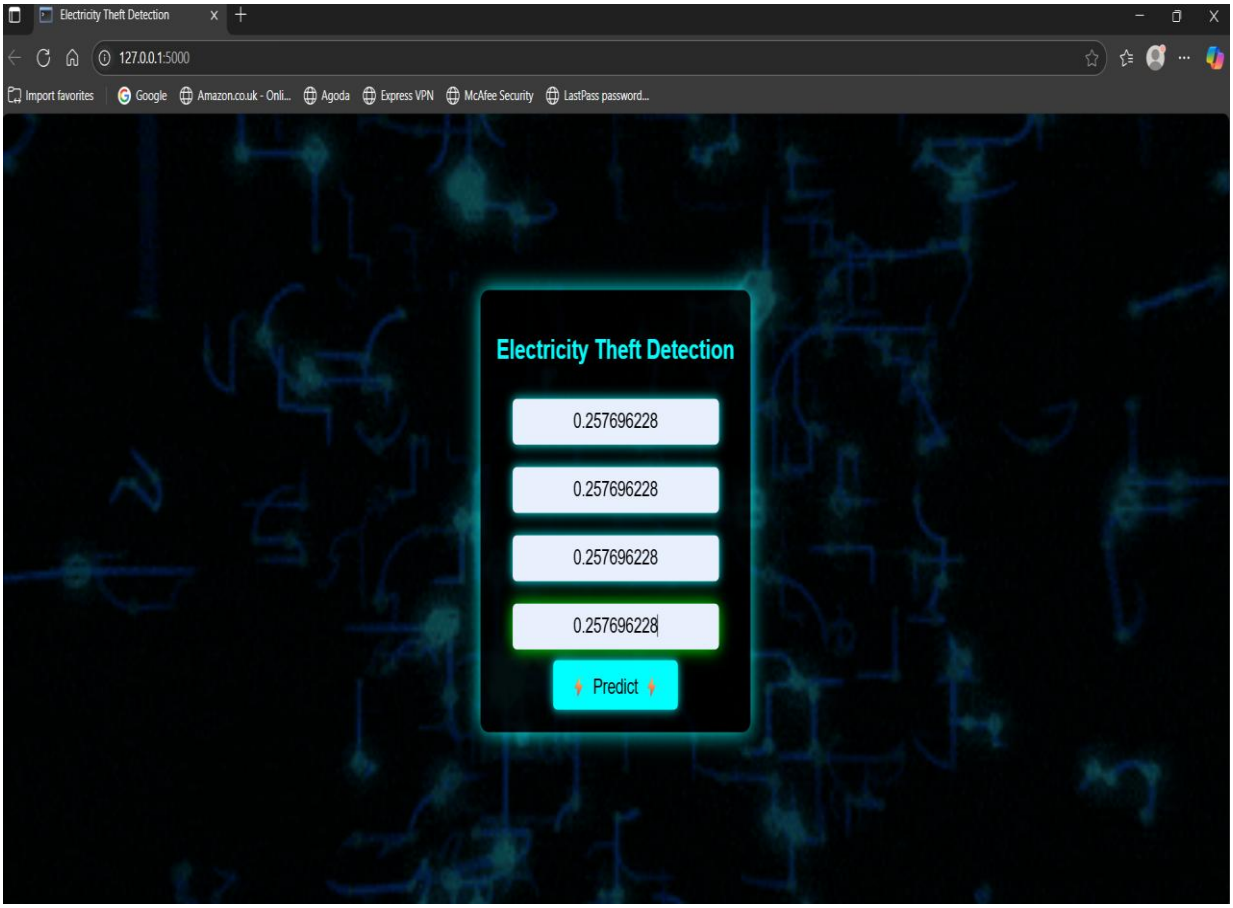


Fig 4.4.4: Values Indicating Electricity Theft

The figure depicts the entered data in the textboxes will be displaying the final output as the electricity theft has happened. After clicking on the predict button it gives the result as the THEFT. Indicating some electricity theft has happened.

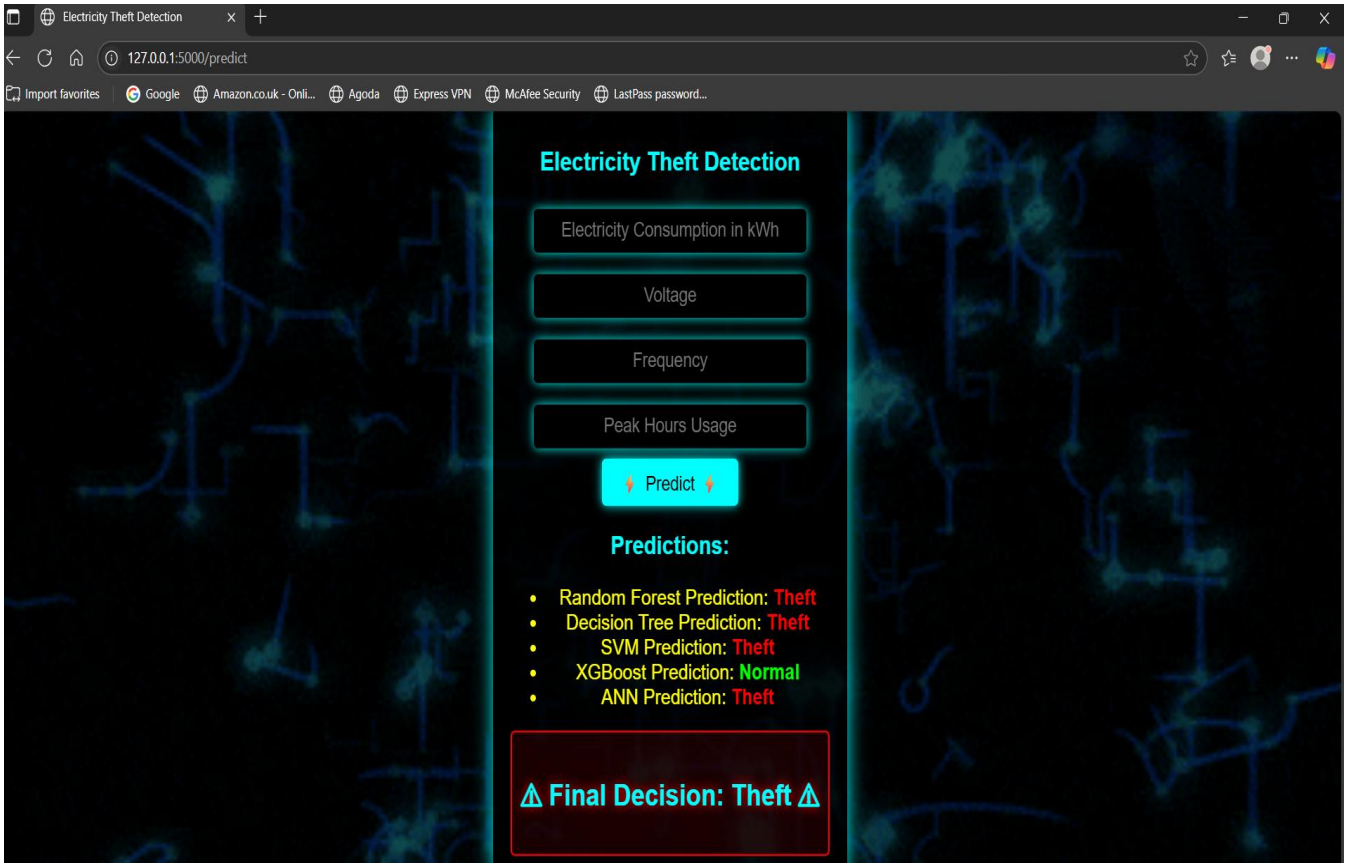


Fig 4.4.5: Electricity Theft Detected

This image shows the final prediction page of the Electricity Theft Detection System after input values are submitted. Four out of five ML Algorithms (RF, DT, SVM, and ANN) have flagged the instance as **Theft**, while only XGBoost predicts **Normal**. Based on majority voting, the system concludes and displays **"Final Decision: Theft"** highlighted in red. This alert serves as a warning to the admin or utility provider about potential electricity fraud.

.

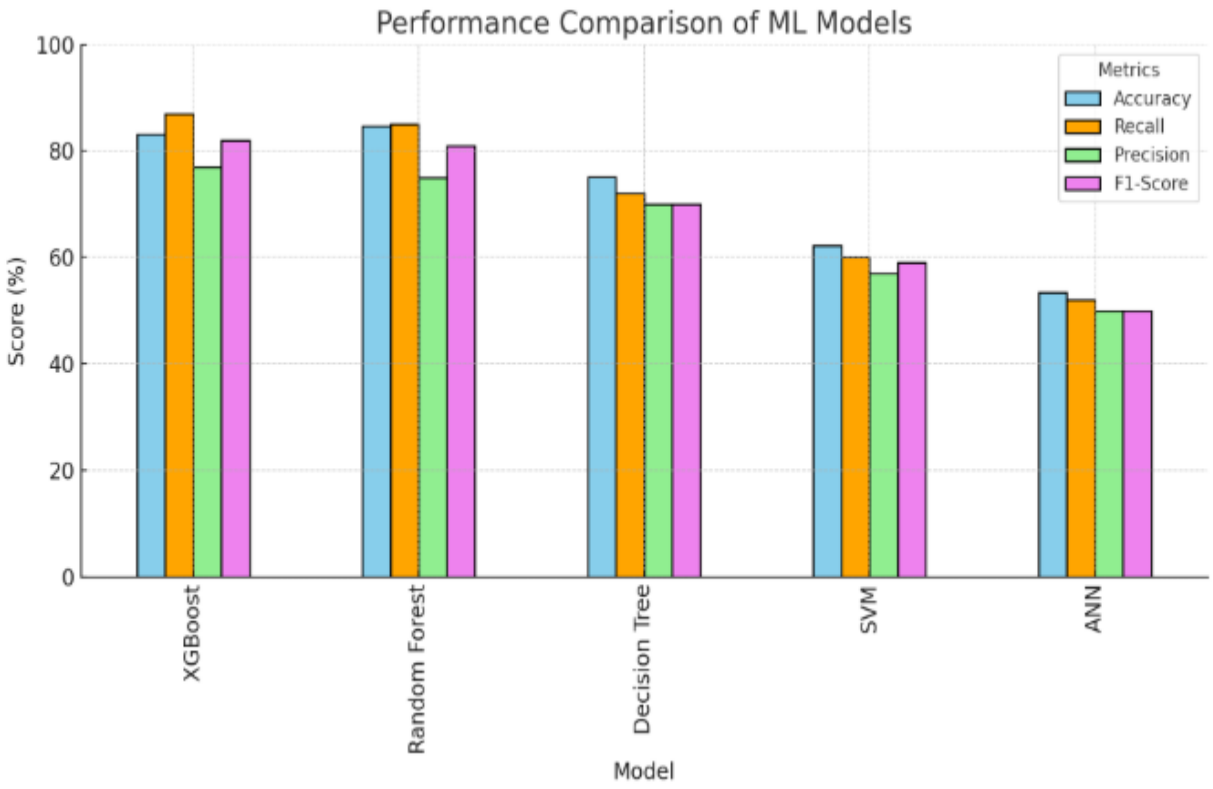


Figure 4.4.6: Evaluation of ML Algorithms

This bar graph provides a visual performance comparison of five Algorithms . Each bar is color-coded to represent one of the four metrics, making it easy to compare the model's side by side. XGBoost and Random Forest clearly stand out with high values across all metrics, showing they are highly effective for electricity theft detection. The Decision Tree shows average performance with slightly lower scores, while SVM and ANN fall short in accuracy and consistency. The ANN model, in particular, performs the worst, indicating poor learning and prediction ability. Among all metrics, recall is critical for identifying theft, and XGBoost achieves the best recall value. Overall, the graph justifies why XGBoost is the most reliable model in this system.

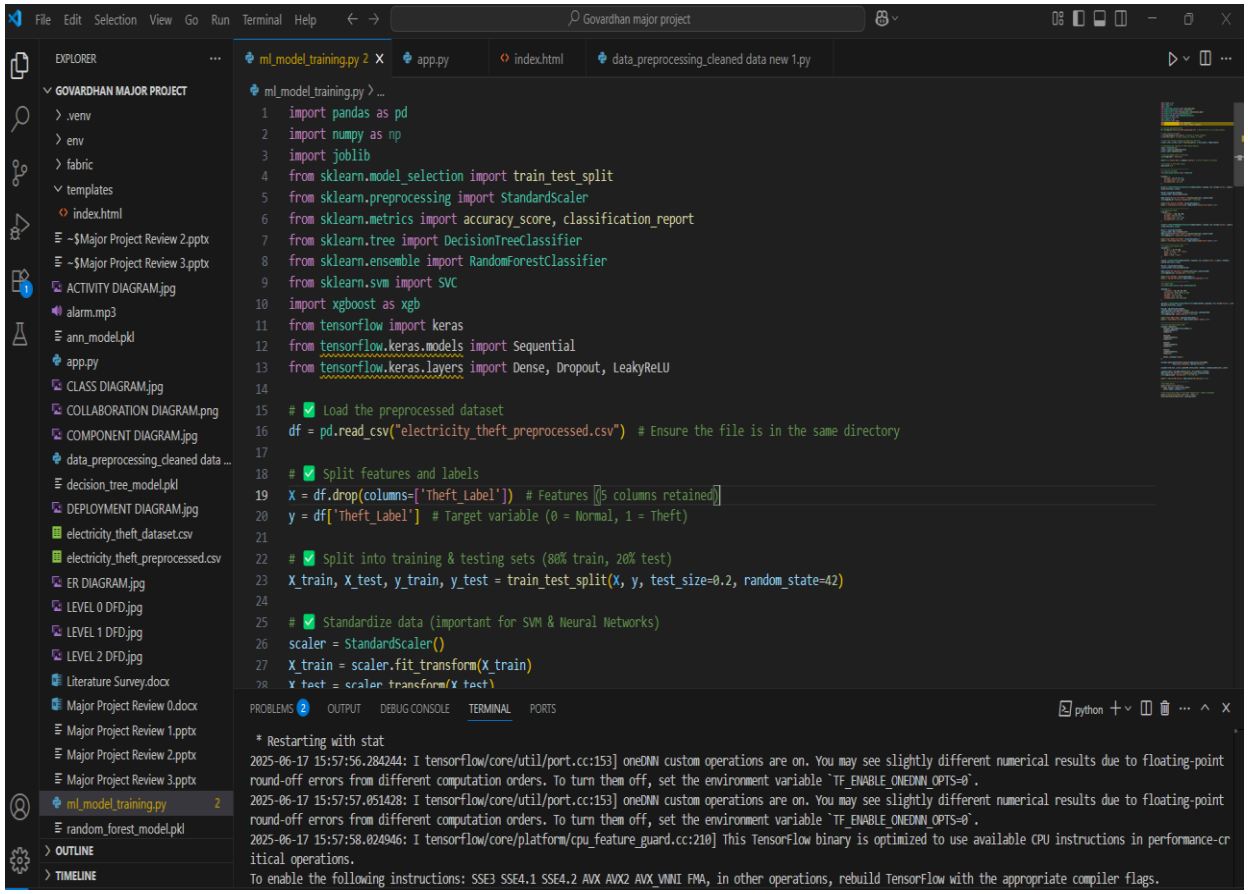


Fig 4.4.7: Image Indicating the Training of ML Algorithms

The training of machine learning algorithms used to identify electrical theft is shown in the figure. Following a successful training process, the model will be able to forecast the result based on the user-inputted values in the fields that were noted in the smart meter's data.

4.5 SAMPLE CODE:

app.py

```
import numpy as np

import pandas as pd

import joblib

import threading

from playsound import playsound

from flask import Flask, request, jsonify, render_template



scaler = joblib.load("scaler.pkl")

rf_model = joblib.load("random_forest_model.pkl")

dt_model = joblib.load("decision_tree_model.pkl")

svm_model = joblib.load("svm_model.pkl")

xgb_model = joblib.load("xgboost_model.pkl")

ann_model = joblib.load("ann_model.pkl")



app = Flask(__name__)
```

```
    playsound("alarm.mp3")
```

Index.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Electricity Theft Detection</title>
    <style>
        body {
            font-family: Arial, sans-serif;
            background: black;
            color: white;
            text-align: center;
            margin: 0;
            padding: 0;
            overflow: hidden;
        }

        .container {
```

```
        position: absolute;
        top: 50%;
        left: 50%;
        transform: translate (-50%, -50%);
        background: rgba (0, 0, 0, 0.8);
        padding: 20px;
        border-radius: 10px;
        box-shadow: 0 0 20px cyan;
    }
```

data_preprocessing_cleaned_data new.py

```
scaler = MinMaxScaler ()
df[['Consumption_kWh', 'Voltage', 'Frequency', 'Peak_Hours_Usage']] = scaler.fit_transform(
    df[['Consumption_kWh', 'Voltage', 'Frequency', 'Peak_Hours_Usage']]
)


# Handle Imbalanced Data using SMOTE
X = df.drop(columns=['Theft_Label'], errors='ignore')
y = df['Theft_Label']
```

CHAPTER -5

5.1 TESTING

TEST CASE DESCRIPTION:

The process of testing is called error detection. Software quality and dependability assurance both heavily rely on testing. The testing results are also used in later maintenance processes.

Psychology of Testing

Testing done show error-free. Finding any potential bugs in the application is the main goal of the testing process. Therefore, the goal of testing should be to demonstrate that a program does not function, not to demonstrate that it does. The process of running a software with the goal of identifying faults is called testing.

Testing Objectives

Finding as many errors as possible, methodically, and with the least amount of time and effort is the main objective of testing. Running a program with the intention of finding mistakes is the formal definition of testing.
A test is deemed successful if it discovers a mistake that hasn't been discovered yet. A test case is deemed outstanding if it has a high probability of detecting errors, if any exist.

- It is not possible to detect possible errors with the tests alone.
- In general, the program meets reliable and high-quality standards.

5.2 Types of Testing

We have the idea of layers of testing to find errors that are present in various stages.

5.2.1 System Testing

Finding errors is the aim of testing. That's why test cases are created. Code testing is one method used for system testing.

5.2.2 Code Testing

This strategy examines the logic of the program. To employ this method, we generated some test data that resulted in the testing of every path or the execution of the instructions for every program and module. Systems are neither produced as wholes nor evaluated as separate systems. To ensure that the coding is perfect, two distinct testing procedures are performed, or, for that matter, are performed on all systems.

5.2.3 Unit Testing

Unit testing concentrates verification efforts on the module, which is the smallest unit of software. Testing is done to find problems inside the module's boundaries using the process specifications and the detailed design. Before the integration testing can occur, every module has to pass the unit test. Every service in this project may be considered a module. Numerous modules are available, including those for image uploading, editing profiles, searching names, viewing profiles, and logging in. Every module has been put to the test with various input sets. When creating the module and completing the development process to ensure that every module functions flawlessly. When the user accepts the input, it is verified. The creator of this application tests the applications in the system. The modules and procedures that are put together and combined to provide a certain function are known as software units in a system. To find issues, unit testing is initially performed on separate modules. This makes mistake detection possible. This mistake was formerly prevented by the interaction between the components.

5.2.4 Link Testing

Link testing examines how each module integrates into the system, not the software itself. The compatibility of each module is the main issue. The programmer runs tests on modules that have varying lengths, types, and other design characteristics.

5.2.5 Integration Testing

Integration testing must be done after unit testing. Checking that modules can be merged correctly is the aim here, with a focus on checking interfaces between modules. Module interactions are being tested since this testing activity may be viewed as evaluating the design. In this project, the primary system is formed by combining all of the components. I tested if the integration affected any of the services' functionality while combining all the modules by providing various input combinations that allowed the two services to function flawlessly before integration.

5.2.6 System Testing

This tests the software system as a whole. The requirements document serves as the process's reference document, and the objective is to determine if the program satisfies those requirements. Here, the complete "ATM" has been tested against the project specifications to see whether or not all of the requirements have been met.

5.2.7 Acceptance Testing

Acceptance To show that the program is operating successfully, a test is run using actual client data. Here, the emphasis of testing is on the system's exterior behavior rather than the underlying logic of the program. The purpose of test cases is to exercise as many properties of an equivalency class as possible at once. A crucial stage in the creation of software is testing. It is the process of identifying mistakes and incomplete tasks as well as a thorough verification to ascertain whether the goals are fulfilled and the needs of the user are satisfied.

5.2.8 White Box Testing

Using this unit testing approach, each unit is evaluated in-depth at the statement level in order to identify the greatest number of problems that might occur. I verified each line of code one by one, making sure that each statement was run at least once. Glass Box Testing is another name for white box testing. For the purpose of verifying every conceivable combination of execution pathways through the code at each module level, I have created a list of test cases and sample data.

5.2.9 Black Box Testing

Rather to delving into specifics at the statement level, this testing approach treats a module as a single entity and evaluates it at the interface and interactions with other modules. In this case, the module will be thought of as a block box that can receive input and produce output. Other modules get the output for a specific set of input combinations. A computer network utility called Ping is used to determine if a certain host is accessible via an IP network. Sending ICMP packets to the intended host and listening for ICMP responses is how Ping operates. Ping calculates the roundtrip time and packet loss rate between sites using interval timing and response rate.

Testing Cycle

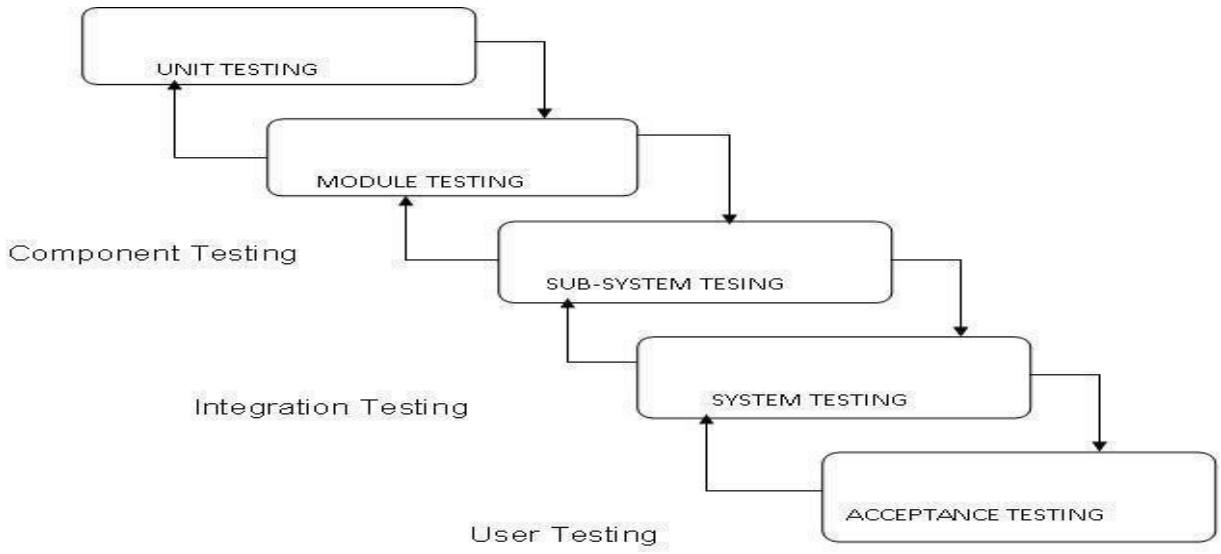


Fig 5.2: Testing Cycle

5.3 TEST CASES TABLES:

Test Case ID	Test Scenario	Test Description	Input Data	Expected Output	Status
1	Load Training Dataset	Check if training dataset loads correctly	Electricity Theft.csv	Dataset preview displayed with all rows and column headers	Pass
2	Load Test Dataset	Check if test dataset loads correctly	test.csv	Test dataset loaded without class labels	Pass
3	Preprocess Dataset	Convert non-numeric data to numeric	Raw dataset	Dataset with integer IDs for categorical fields	Pass
4	Train XGBoost Model	Train model using XGBoost only	Preprocessed dataset	Training complete, Accuracy displayed (e.g., 94%)	Pass
5	Train XGBoost + RF Model	Train using XGBoost combined with Random Forest	Preprocessed dataset	Accuracy displayed (expected ~100%)	Pass
6	Train ANN + SVM Model	Train using combined ANN with SVM	Preprocessed dataset	Accuracy displayed (expected ~99%)	Pass
7	Train Random Forest Alone	Train Random Forest	Preprocessed dataset	Accuracy displayed (expected ~94%)	Pass
8	Train DT Alone	Train DT	Preprocessed dataset	Accuracy displayed (expected ~96%)	Pass
9	Predict Electricity Theft	Use Them to classify test data as theft or not	Test dataset	Outputs: "record detected as ENERGY THEFT" or "NOT detected ."	Pass

10	Accuracy Comparison Graph	Check if graph compares accuracy of all models	All model results	Graph with CNN, CNN-RF, CNN-SVM, RF, SVM – with respective metrics	Pass
11	Run Button Functionality	Validate functionality of run.batto launch system	Double-click run.bat	UI launches without error	Pass
12	File Upload Validation	Check invalid file type or corrupted data upload	Non-CSV file / Corrupted file	Error message displayed, file not accepted	Pass
13	Handle Missing Values	Verify preprocessing handles missing/null values correctly	Dataset with empty cells	Cleaned dataset with no nulls	Pass
14	Output Format Validation	Check if prediction results follow the expected message format	Test sample	Message: "[test data] - record detected as ENERGY THEFT"	Pass

CHAPTER-6

CONCLUSION & FUTURE ENHANCEMENT

6.1 CONCLUSION:

The increasing prevalence of electricity theft poses a serious challenge to modern power distribution systems, leading to economic losses and grid instability. In this project, a data-driven solution was developed using advanced machine learning models to detect fraudulent consumption behavior with high precision. By leveraging smart meter data and historical consumption patterns, the system effectively identifies anomalies that indicate theft, making it a valuable tool for utility providers seeking intelligent monitoring solutions.

Throughout the project, multiple supervised Artificial Neural Networks, Random Forest, Support Vector Machine, XGBoost, and Decision Tree learning methods were used and assessed. The model that performed the best among them was XGBoost, which provided a good balance between execution time, accuracy, and recall. The models' efficiency and capacity for generalization were greatly enhanced by the application of strategies including features scaling, SMOTE for managing unbalanced input, and hyperparameter tuning.

The system architecture was designed to ensure seamless integration with real-time dashboards via Flask, enabling utility operators to view predictions and receive alerts in case of detected theft. Each component, from preprocessing to final decision-making, was modularly developed for easy scalability and maintainability. This ensures that the solution remains effective.

In conclusion, the project successfully demonstrates a robust, intelligent, and scalable approach to electricity theft detection using machine learning. The model not only aids in identifying suspicious consumption patterns but also minimizes false positives, thereby ensuring reliability. With further enhancements like real-time data streaming, geographical mapping, and alert automation, this solution can be extended into a full-fledged smart grid security platform. This work lays a strong foundation for future innovation in energy fraud analytics and smart grid resilience.

6.2 FUTURE ENHANCEMENT:

The electricity theft detection system developed in this project can be enhanced further to ensure greater scalability, real-time responsiveness, and accuracy in dynamic smart grid environments. As electricity networks evolve with the integration of IoT, AI, and big data analytics, the following enhancements can significantly boost the system's resilience and effectiveness:

- **Real-Time Detection with IoT Integration:**

Future improvements can include live data streaming from smart meters using IoT devices. This allows the system to detect theft instantaneously and send alerts to utility providers, making the system more proactive rather than reactive.

- **Deployment of Edge-Based Detection:**

By shifting parts of the detection logic to edge devices or gateways, electricity theft detection can occur locally without relying heavily on central servers. This reduces latency and ensures faster response, especially in rural or bandwidth-limited regions.

- **Incorporating Explainable AI (XAI):**

To build trust and interpretability, future systems can integrate XAI methods, which help explain why a particular user was flagged as suspicious. This would help utility companies validate decisions and refine policies without relying solely on black-box models.

- **Advanced Visualization and Geospatial Mapping:**

A future enhancement could involve integrating geospatial dashboards that map detected theft cases by location. This helps utilities understand regional theft patterns and allocate resources like inspection teams more strategically.

REFERNCES

1. Jain, A. & Kumar, R. (2023). Detection of Electricity Theft in Smart Grids Using ML Approaches. *International Journal of Engineering Research & Technology*, 12(3), 214–219.
2. Alsheikh, M., Abdallah, A. E., & Hossain, S. I. (2022). Smart Meter Framework for Detecting Electricity Theft. *Energy Reports*, 8, 1789–1799.
3. Kumar, S. & Patel, N. (2022). Evaluation of Machine Learning Classifiers for Power Theft Detection. *IEEE Access*, 10, 65789–65797.
4. Das, R. & Gupta, A. (2021). Using XGBoost for Anomaly Detection in Energy Usage. In *Smart Innovation Series* (Vol. 123, pp. 91–101). Springer.
5. Chen, Y. et al. (2023). Monitoring Electricity Theft in Real-Time via Smart Grids. *IEEE Internet of Things Journal*, 10(2), 1221–1232.
6. Rajasekaran, S. & Ghosh, P. (2022). Hybrid ML Techniques for Energy Theft Identification. *Journal of Electrical Systems and Information Technology*, 9(4).
7. Mohammed, A. & Ahmed, M. (2021). Deep Learning Models for Detection of Power Theft. *International Journal of Computer Applications*, 184(18), 12–19.
8. Alsharif, T. & Al-Ghamdi, H. (2021). Smart Grid Intrusion Prevention Using ML-Based Detection. *Computers & Electrical Engineering*, 94, 107307.
9. Verma, P. & Jha, N. K. (2020). Preventing Electricity Theft Using Big Data Analytics in Smart Grids. *Procedia Computer Science*, 167, 2691–2699.
10. Jindal, R. & Bedi, H. (2020). Power Theft Detection Using Random Forest Classification. *International Journal of Advanced Computer Science and Applications*, 11(6), 341–346.
11. Sheik, M. & Ilyas, K. (2020). Smart Meter Fraud Analysis Through Data-Driven Techniques. *International Journal of Electrical Power & Energy Systems*, 115, 105420.
12. Wang, L. & Liu, J. (2019). ML Use Cases in Smart Grid Environments: A Survey. *Renewable and Sustainable Energy Reviews*, 113, 109292.
13. Srivastava, G. et al. (2021). A Security Architecture for Smart Grids Using AI. *IEEE Transactions on Industrial Informatics*, 17(3), 1948–1955.

14. Sharma, N. & Mathur, A. (2020). Classification of Power Usage Patterns Using Artificial Neural Networks. *International Journal of Innovative Technology and Exploring Engineering*, 9(7), 1015–1019.

15. Singh, B. & Rani, M. (2021). Analysis of Power Grid Vulnerability and Methods for Theft Detection. *Journal of Electrical and Power Engineering*, 11(2), 23–29.

16. Patel, M. & Mehta, S. (2022). Review on Electricity Theft Detection with Smart Meter Analytics. *International Journal of Electrical and Computer Engineering*, 12(1), 45–52.

17. Jha, V. R. & Ramesh, K. (2020). ML Approaches for Identifying Non-Technical Losses in Distribution Systems. *International Journal of Scientific & Engineering Research*, 11(9), 198–204.

18. Ghosh, D. & Sinha, P. (2021). Identifying Power Theft Using Decision Tree and SVM Models. *International Journal of Computer Sciences and Engineering*, 8(10), 110–116.